

IPC to BNS Mapper

Docker Deployment Documentation

Author	Aritram Dutta
Organisation	GreenAI Services Private Limited
Product	Bidhi Lab — Legal Tech Platform
Stack	Next.js 15 · n8n · Docker · TypeScript
Scope	Docker containerisation & deployment

1. Project Overview

The IPC to BNS Mapper is a legal technology solution developed to automatically identify Indian Penal Code (IPC) sections from user-provided text or PDF documents and map them to the corresponding Bharatiya Nyaya Sanhita (BNS) sections. The solution is designed to be integrated into the **Bidhi Lab** section of the GreenAI Services Private Limited platform.

2. System Architecture

The Docker package runs two containerised services that communicate over Docker's internal network:

Service	Technology	Port	Responsibilities
Frontend	Next.js 15 · React 19 TypeScript · Tailwind CSS	3000	Accept text / PDF input Display IPC → BNS mappings Show highlighted annotations
Backend (n8n)	n8n Workflow Automation JavaScript Code Nodes PDF Processing Nodes	5678	Receive text or PDF Extract & detect IPC sections Apply BNS mapping Return JSON to frontend

3. Project Structure

```
ipc-bns-docker/  
├── docker-compose.yml  
├── frontend/  
│   ├── Dockerfile  
│   ├── src/  
│   ├── public/  
│   ├── package.json  
│   └── ...  
├── workflow/  
│   └── ipc-bns-workflow.json  
└── n8n_data/
```

4. Data Flow

Step	Actor / Component	Action
1	User	Initiates a request via the web browser.
2	Frontend (Next.js)	Sends a POST request to the n8n webhook endpoint.
3	n8n Webhook	Receives the request and triggers the automation workflow.
4	Text / PDF Processing	Extracts raw text from uploaded documents.

5	IPC Extraction	Detects IPC section references using pattern matching.
6	IPC → BNS Mapping	Looks up corresponding BNS sections from the mapping database.
7	JSON Response	Structured result returned to the frontend.
8	Frontend Display	Annotated text and mappings rendered in the browser.

5. Docker Configuration

5.1 Frontend Container

Container Name	ipc-bns-frontend
Exposed Port	3000
Base Image	Node.js
Env Variable	N8N_WEBHOOK_1=http://n8n:5678/webhook/analyze-document

5.2 n8n Backend Container

Container Name	ipc-bns-n8n
Docker Image	n8nio/n8n
Exposed Port	5678
Webhook Path	POST /webhook/analyze-document

6. Deployment Steps

6.1 Prerequisites

Install Docker Desktop and Docker Engine, then verify:

```
docker --version
docker compose version
```

6.2 Start the Application

Navigate to the project folder and start both services:

```
cd ipc-bns-docker
docker compose up --build -d
```

6.3 Verify Running Containers

```
docker ps
```

Expected containers:

- **ipc-bns-frontend**
- **ipc-bns-n8n**

6.4 Access URLs

Service	URL
Frontend App	http://localhost:3000
n8n Dashboard	http://localhost:5678

7. Workflow Import Procedure

1. Open the n8n dashboard at **http://localhost:5678**
2. Navigate to **Workflows**
3. Click **Import Workflow**
4. Select the file: **workflow/ipc-bns-workflow.json**
5. Click **Activate** to enable the workflow

Active webhook endpoint after import:

```
POST http://localhost:5678/webhook/analyze-document
```

8. API Communication

8.1 Text Request (JSON)

```
{
  "text": "IPC Section 302"
}
```

8.2 PDF Request (multipart/form-data)

```
Content-Type: multipart/form-data
file=document.pdf
```

8.3 Response Format

```
{
  "detected_ipc_sections": ["IPC Section 302"],
  "mapped_bns_sections":  ["BNS Section 103"],
  "explanations":         ["IPC Section 302 maps to BNS Section 103"],
  "annotated_text":       "...",
  "raw_text":              "...",
}
```

9. Troubleshooting

Fetch Failed

Cause: Incorrect webhook URL in environment variables.

Fix:

```
N8N_WEBHOOK_1=http://n8n:5678/webhook/analyze-document
```

n8n Stuck on Startup

Cause: Container may have crashed or volume issue.

Fix:

```
docker logs ipc-bns-n8n
```

Verify Environment Variable

Cause: Confirm the env var is correctly set inside the running container.

Fix:

```
docker exec ipc-bns-frontend printenv | grep N8N
```

10. Cloud Deployment Options

The Docker package can be deployed to any container-capable platform without modifying source code:

AWS EC2	AWS ECS / Fargate
Azure Container Apps	Google Cloud Run
Railway	Render
Private Linux VPS	Any Docker-compatible host

11. Future Enhancement — Phase 2

Phase 2 plans to replace the n8n backend with a dedicated API service while keeping the frontend unchanged:

Component	Phase 1 (Current)	Phase 2 (Planned)
Backend Engine	n8n Workflow Automation	Node.js + Express.js
Language	JavaScript (Code Nodes)	TypeScript API
Deployment Size	Larger (n8n image)	Smaller (custom service)
Performance	Good	Faster
Scalability	Standard	Enterprise-grade
Frontend Changes	—	None required